# Contents

**1 Translating molecules for computers** **3**

1.1 Information in chemical structures . . . . . 3

1.2 Coordinates and molecular geometry . . . . . 4

1.3 Molecules as graphs . . . . . 7

1.4 Line notations . . . . . 8

1.5 Identifiers and chemical records . . . . . 10

1.6 Bit vectors and molecular fingerprints . . . . . 13

1.7 Summary . . . . . 16

# 1. Translating molecules for computers

## 1.1. Information in chemical structures

Chemists routinely communicate molecular structure using drawings. A skeletal formula can encode a considerable amount of information in a compact form because we have learnt conventions for interpreting atoms, bonds, functional groups and stereochemistry. Consider a simple example of proline (Fig. 1.1) and what information we implicitly capture about its structure.

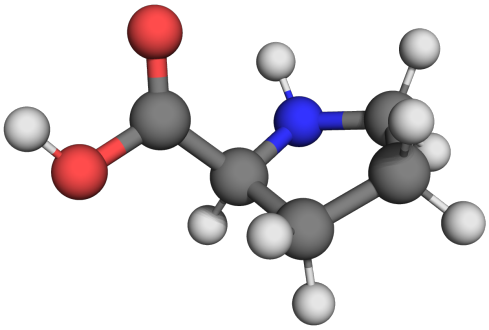


**Figure 1.1.** Molecular structure of proline.

Computers do not interpret chemical drawings in the same way. To store, search, compare and analyse molecules computationally, chemical information must first be represented in a form that can be processed by computer algorithms.

**Main question:** how should we represent a molecule to a computer?

Throughout this chapter, we will encounter several ways of representing molecules digitally. These range from three-dimensional atomic coordinates to text

strings, identifiers and numerical fingerprints. Rather than asking which representation is 'best', we will ask **what chemical information each representation contains**, what it discards, and what it allows us to do.

The information required by a computer depends on the task. A calculation of molecular geometry requires the positions of atoms in three-dimensional space. A substructure search requires information about atoms and their connectivity. Searching a database for a particular chemical requires a consistent identifier. Comparing the structural similarity of thousands of compounds requires a representation that can be compared efficiently. Consequently, the same molecule may be represented in several different ways within a single workflow.

**For each task below, consider what information must be retained:**

- calculate the distance between two atoms;
- determine whether a molecule contains a substructure;
- find a compound in a database;
- compare a molecule with one million other structures.

## 1.2. Coordinates and molecular geometry

One of the most direct ways of describing a geometry is to specify the position of every atom in three-dimensional space, often in a simple `.xyz` file (Lst. 1.1). The first line contains the number of atoms, followed by molecule name, and its coordinates. Other filetypes (*e.g.*, `.mol2`) might also include bonding information.

```
17
Proline
   H       -0.13097       -1.07655        1.63572
   C       -0.15726       -0.63902        0.60818
   C       -1.20401       -1.38526       -0.20437
   O       -1.15189       -1.78305       -1.35456
   O       -2.35852       -1.64177        0.45250
   H       -2.96245       -2.10328       -0.12133
   N        1.22788       -0.77675        0.06680
   C        1.69088        0.54717       -0.43489
   H        2.56627        0.82982        0.18087
   H        2.04876        0.49358       -1.48090
   C        0.54048        1.53722       -0.27264
   H        0.08541        1.78163       -1.25139
   H        0.88732        2.50157        0.14048
   C       -0.47304        0.86447        0.64762
   H       -1.51350        1.08299        0.34489
   H       -0.38097        1.24444        1.68280
   H        1.28559       -1.47722       -0.63981
```

**Listing 1.1.** Proline geometry as an `.xyz` file.

A Cartesian representation assigns three coordinates to each atom. Molecular geometries can alternatively be expressed using **internal coordinates**, such as bond

lengths, bond angles, and dihedral angles, in a *Z-matrix*. Quantum-chemical calculations and molecular simulations routinely use representations of this type. However, even such explicit representation might miss important details required for calculations, *e.g.*, overall charge and spin multiplicity (Lst. 1.2).

```
! B3LYP D3BJ def2-TZVP OPT FREQ

* xyzfile 0 1 proline.xyz
```

**Listing 1.2.** An ORCA input file for a geometry optimisation calculation of proline.

Now, consider the two structures of proline in Figure 1.2 – are they the same molecule? A useful molecular representation should not change just because we move or rotate a molecule in space.

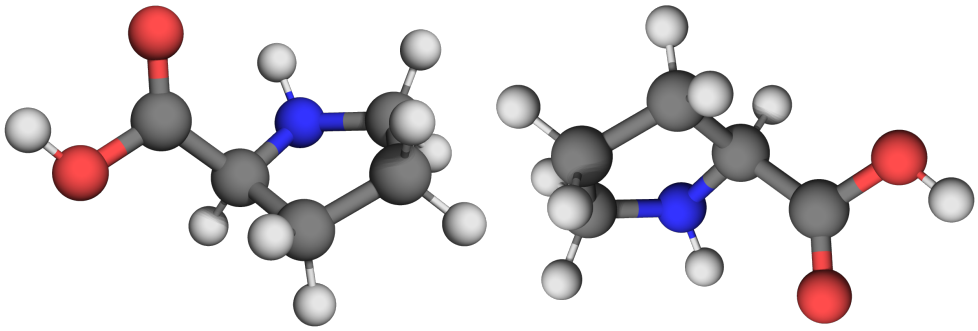


**Figure 1.2.** Two equivalent molecular structures of proline with different atomic coordinates.

One way to retain information about both atomic identity and molecular geometry upon rotation is the **Coulomb matrix**. For atoms with nuclear charges $Z_i$ and Cartesian positions $\mathbf{R}_i$,

$$C_{ij} = \begin{cases} \frac{1}{2} Z_i^{2.4} & \text{for} \quad i = j, \\ \frac{Z_i Z_j}{|\mathbf{R}_i - \mathbf{R}_j|} & \text{for} \quad i \neq j. \end{cases} \tag{1.1}$$

The exponent 2.4 is empirical rather than fundamental: the diagonal term approximates the energy of an isolated atom.

Because the off-diagonal elements depend only on interatomic distances, translating or rotating the molecule does not the change Coulomb matrix. Reordering the atoms permutes the rows and columns of the matrix. An **eigenvector** is a direction that is only scaled when acted upon by a matrix: $\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$, where $\mathbf{v}$ is the eigenvector and $\lambda$ is the corresponding **eigenvalue**. Therefore, while the matrix not invariant to atom ordering, its **sorted eigenspectrum** is:

$$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \ldots, \lambda_N) \qquad \text{where} \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N. \tag{1.2}$$

**Invariant does not mean unique:** the Coulomb matrix eigenspectrum removes dependence on molecular orientation and atom ordering, but different molecular structures can share the same eigenspectrum. It is, therefore, not necessarily a unique molecular identifier.

Consider a water molecule with the following Cartesian coordinates (in Å):

| Atom | $x$ | $y$ | $z$ |
|---|---|---|---|
| O | 0.0000 | 0.0000 | 0.0000 |
| H | 0.9572 | 0.0000 | 0.0000 |
| H | -0.2400 | 0.9273 | 0.0000 |

## 1.3. Molecules as graphs

Three-dimensional representations are essential when the geometry of a molecule matters. For many chemical problems, however, we are primarily interested in which atoms are connected to which (**molecular topology**). This motivates representing a molecule as a **molecular graph** (Fig. 1.3).

A mathematical graph consists of a collection of **vertices** connected by **edges**. Molecular structure maps naturally onto this representation: atoms become vertices and bonds become edges. A molecular graph can contain more information than connectivity alone. Properties can be attached to both atoms and bonds. Such a graph is sometimes described as an **attributed graph**.

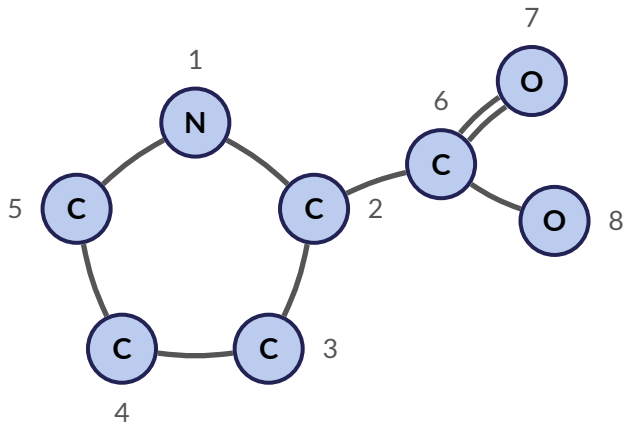



**Figure 1.3.** Hydrogen-suppressed molecular graph of proline. Atomic identity is represented as a vertex attribute and bond order as an edge attribute. The numerical vertex indices are arbitrary.

Once the vertices of a molecular graph have been assigned indices, the connectivity of the graph can itself be represented numerically. For a simple, unweighted molecular graph, the **adjacency matrix** $\mathbf{A}$ is defined as

We could define different numerical values for edge attributes.

$$A_{ij} = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are connected,} \\ 0, & \text{otherwise.} \end{cases} \tag{1.3}$$

## 1.4. Line notations

Molecular graphs provide a convenient abstract representation of chemical connectivity, but explicitly storing and displaying a graph is not always practical. An alternative is to encode the molecular graph as a sequence of characters. Such **line representations** allow molecular structures to be stored, searched and exchanged using ordinary text. The most widely encountered example is the **S**implified **M**olecular **I**nput **L**ine **E**ntry **S**ystem, or **SMILES**.

### 1.4.1. SMILES syntax

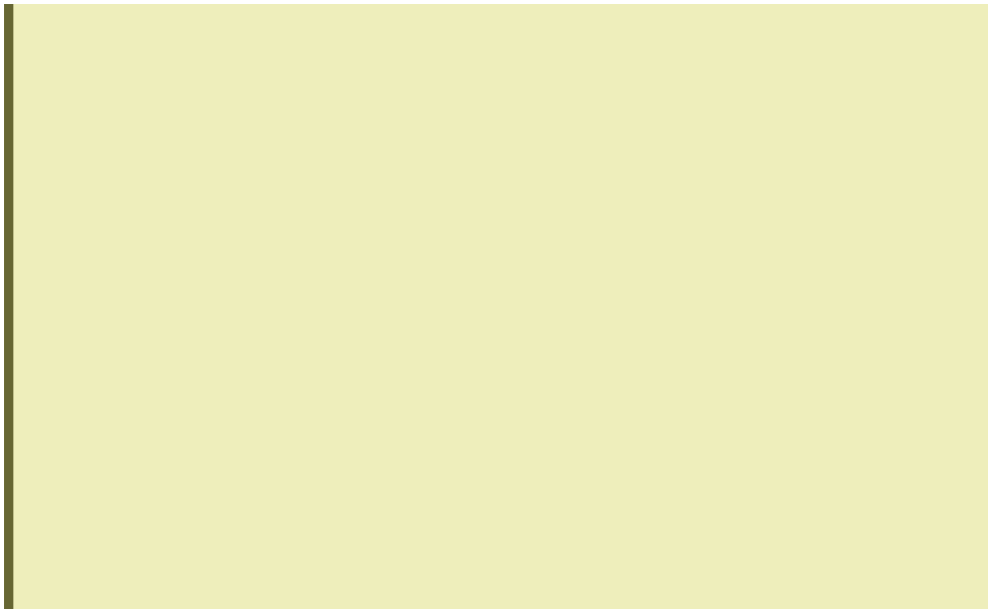
Atoms are represented using their element symbols. Adjacent atoms in the string are connected by a bond. Hydrogen atoms attached to typical organic atoms are normally omitted and inferred from standard valence rules. Single bonds are normally omitted, so for ethanol:

$$\mathrm{C{-}C{-}O} \qquad \longrightarrow \qquad \texttt{CCO} \tag{1.4}$$

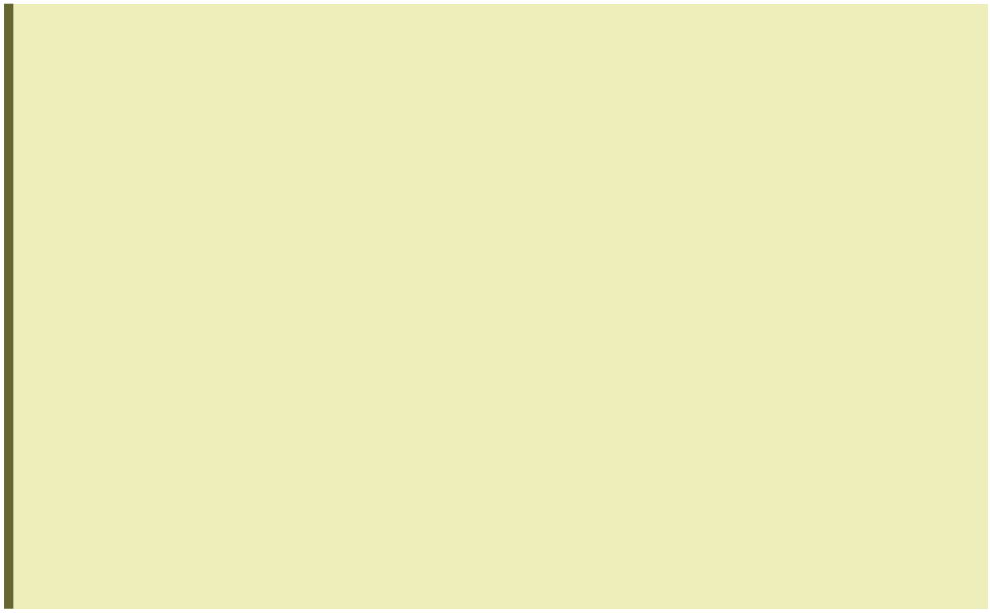
Double and triple bonds are written explicitly using an equals sign (`O=C=O` is carbon dioxide) and a hash symbol (`C#C` is acetylene). A branch in the molecular graph is written inside parentheses following the atom (`CC(=O)N` is acetamide). Tetrahedral stereochemistry can be included using `@` and `@@`, interpreted relative to the order of neighbouring atoms in the string. Double-bond stereochemistry can be represented using directional bond symbols.

A cyclic molecular graph cannot be represented simply by walking along a sequence of atoms without eventually reconnecting two vertices. SMILES indicates this **ring closure** using matching numerical labels. Aromatic atoms can be represented either as Kekulé structure or with lower-case letters.

**The ring number does not represent an atom.** Matching ring labels indicate the edge connecting two atoms encountered during the graph traversal.



A molecular graph can usually be traversed in more than one way. Consequently, a single molecule can have multiple valid SMILES representations.



For database searching and comparison, having many valid strings for the same molecular graph is inconvenient. A **canonicalisation algorithm** assigns a reproducible ordering to the graph and generates a corresponding canonical SMILES.

**Canonical does not mean universal.** Canonical SMILES are generated algorithmically. Different software packages can employ different canonicalisation procedures, so the same molecule need not have one universally defined canonical SMILES. These can even vary between software versions!

Several related line notations extend or modify the ideas used in SMILES. **SMARTS** describes a molecular pattern rather than one specific molecule. It is used for substructure searching, allowing questions such as whether a molecule contains an alcohol, amide or aromatic nitrogen atom. **SMIRKS** encodes a chemical transformation by specifying patterns in the reactants and the corresponding patterns in the products. **SELFIES** is a string representation designed so that every valid SELFIES string corresponds to a molecular graph permitted by its underlying rules. This makes SELFIES useful in generative machine learning, where arbitrary modification of a SMILES will often produce an invalid structure.

## 1.5. Identifiers and chemical records

SMILES strings can also contain special characters that are inconvenient when structures are passed between different pieces of software or used as search terms. More fundamentally, the existence of multiple valid SMILES representations makes them inconvenient as universal chemical identifiers for cross-referencing structures from different sources. For searching databases and linking chemical information between different sources, it is useful to have a standardised molecular identifier.

### 1.5.1. The International Chemical Identifier (InChI)

To address these problems, IUPAC developed the **International Chemical Identifier (InChI)**. InChI is a structure-based, non-proprietary, open-source, and standardised identifier in which different types of chemical information are organised hierarchically into layers. This makes InChI particularly useful when chemical information from different sources needs to be linked or incorporated into **FAIR** (**f**indable, **a**ccessible, **i**nteroperable, and **r**eusable) datasets.

**An InChI is organised into layers of chemical information.** An InChI begins with a version identifier and molecular formula, followed by additional layers describing features such as connectivity, hydrogen atoms, charge, stereochemistry, and isotopic composition. Layers are included only where the corresponding information is required to describe the structure.

The ability to read and parse InChI by humans was of little importance to the developers. You are not expected to construct an InChI manually. You should, however, be able to recognise the organisation of an InChI and identify where important types of chemical information are stored (Fig. 1.4).

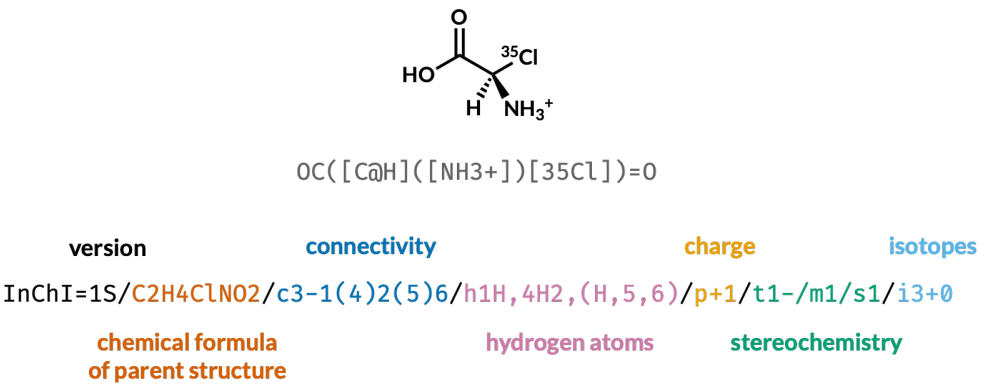



**Figure 1.4.** An example chemical structure with its corresponding InChI. Note how each layer gives more and more chemical information.

### 1.5.2. Search engine and indexing: the InChIKey

Although InChIs provide standardised identifiers, they can have varying lengths and contain characters that are inconvenient for search engines, database indexing and filenames. For this reason, a second identifier, the **InChIKey**, can be generated from the InChI. An InChIKey is a hash of an InChI that produces a compact 27-character identifier containing uppercase letters and hyphens (Fig. 1.5).

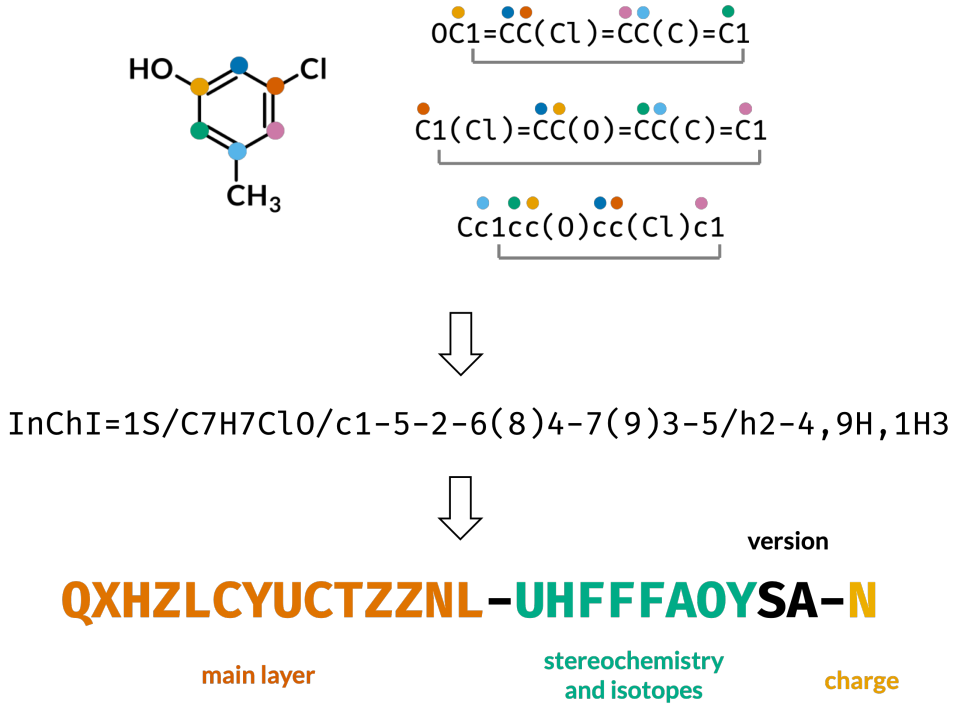
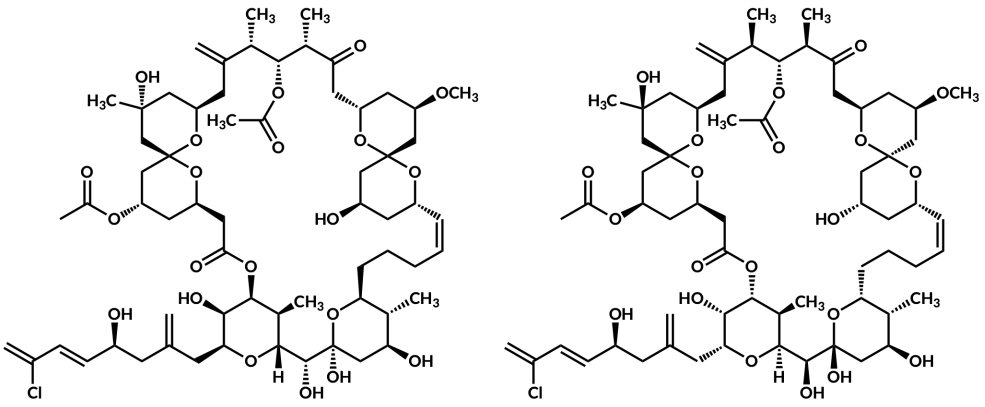



**Figure 1.5.** Different molecular representations of the same molecule: non-unique SMILES, standardised InChI and its corresponding InChIKey.

> Hashing is a one-way operation. The same InChI will reproducibly generate the same InChIKey, but it is not possible to recover the complete molecular structure from the InChIKey alone. While they are convenient filenames, it is wise to keep a 'dictionary' mapping them to structural information.

The InChIKey is divided into blocks. The first block represents information associated primarily with connectivity, while subsequent parts encode information associated with stereochemistry, isotopic composition and protonation.

Compressing an InChI into a much shorter InChIKey necessarily loses information. This creates the possibility of a *hash collision*, in which different InChIs generate the same InChIKey. While the `SHA-256 hash` was chosen to minimise collisions of InChIKeys, they are still possible – especially for large structures. Consider two diastereomers of Spongistatin I, a molecule with 26 stereocentres, which have different InChI but the same InChIKey (Fig. 1.6). Such collisions are incredibly rare, but it is prudent to check they have not happened in our datasets!



`ICXJVZHDZFXYQC-RAZYNMGUSA-N`

**$2^{26}$ = 67,108,864 diastereomers!**

**Figure 1.6.** Two diastereomers of Spongistatin I with the same InChIKey.

## 1.6. Bit vectors and molecular fingerprints

So far, we have considered representations that aim to describe or identify a molecular structure. For many cheminformatics applications, however, we do not need to reconstruct the complete structure of a molecule. Instead, we may want to answer much simpler questions: does a particular structural feature occur in the molecule, and how similar is one molecule to another? For these tasks, it is useful to convert molecular structures into compact numerical representations known as **molecular fingerprints**.

A **bit vector** is a one-dimensional array containing zeros and ones, for example,

$$\mathbf{x} = (0, 0, 1, 1, 0, 1). \tag{1.5}$$

Each position, or *bit*, can be used to record the presence or absence of some feature of a higher-dimensional object such as a molecule. This might initially seem an unnecessarily crude way of representing chemical structure. However, identifying and comparing substructures directly across very large molecular databases can be computationally expensive. Once molecules have been converted into bit vectors, computers can compare their patterns of zeros and ones extremely efficiently.

> **Molecular fingerprints trade detail for computational efficiency.** A molecular fingerprint encodes selected features of a molecular structure in a compact numerical form. It is generally not possible to reconstruct the complete molecular structure from the fingerprint.

### 1.6.1. Structural keys

One simple way to construct such a bit vector is to define a fixed list of yes/no structural questions. Each question is assigned a position in the bit vector, and the corresponding bit is switched on if the feature is present. A very simple list of such structural questions could be:

- does the molecule contain oxygen?
- does it contain a ring?
- does it contain an aromatic atom?
- does it contain a six-membered ring?
- does it contain a carbonyl group?

**A structural-key fingerprint asks a predefined set of questions about a molecule.** Each structural feature is associated with a particular bit:

$$\text{feature absent} \longrightarrow 0$$
$$\text{feature present} \longrightarrow 1$$

Two molecules sharing many of the encoded structural features will consequently share many 'on' bits, which can be cheaply compared.

One widely used example is the **M**olecular **ACC**ess **S**ystem (**MACCS**) structural key fingerprint. MACCS keys use a predefined collection of 166 structural patterns intended to summarise useful information about molecular structure (Fig. 1.7).

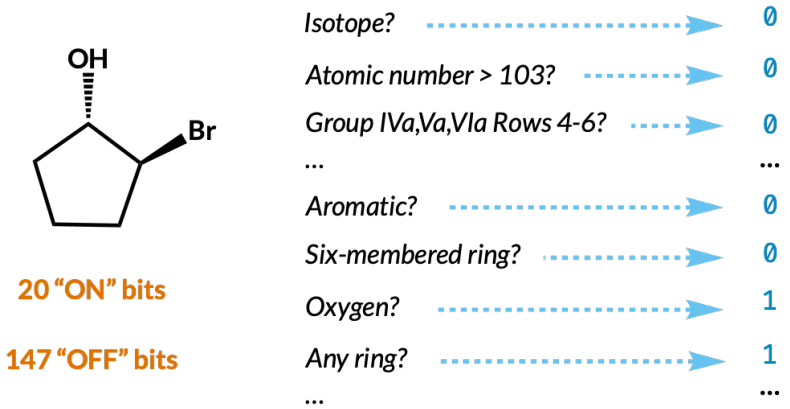



**Figure 1.7.** Example MACCS keys for a simple organic molecule.

Someone must decide in advance which structural patterns are worth encoding. A set of keys designed for one region of chemistry may fail to capture the structural distinctions that matter in another.

### 1.6.2. Circular fingerprints

Rather than deciding in advance which structural patterns are important, we can generate features directly from the molecular graph. **Circular fingerprints** describe the local environment surrounding each atom and progressively expand that environment through neighbouring atoms:

1. assign an initial identifier to each atom;
2. identify the neighbouring atoms and bonds;
3. combine this information to describe progressively larger local environments;
4. convert each environment into a numerical identifier;
5. encode the resulting identifiers in a fingerprint.

The size of an atomic environment is described by its **radius**. At radius 0, the environment describes the atom itself. Radius 1 additionally includes directly bon-

ded neighbours, radius 2 extends to the next shell of atoms, and so forth (Fig. 1.8). Increasing the radius incorporates progressively more of the local molecular environment surrounding each atom.

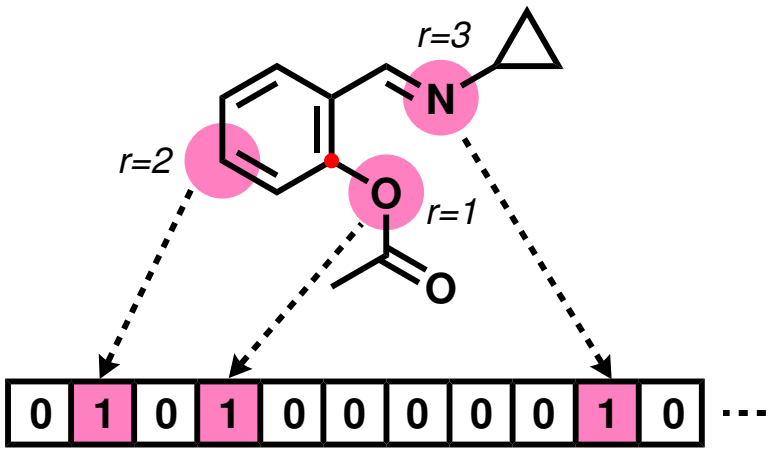



**Figure 1.8.** Conceptual representation of a circular fingerprint.

One of the most widely used approaches to circular fingerprints derives from the Morgan algorithm introduced by the Chemical Abstracts Society (CAS). A more recent development of this idea is the **Extended-Connectivity Fingerprint (ECFP)**. For each atom, the algorithm generates identifiers describing its local molecular environments. These identifiers can then be **hashed**: a numerical operation converts an identifier into an integer, which can subsequently be mapped onto a position in a fingerprint of fixed length through modulo division (Fig. 1.9). If the same structural environment occurs in another molecule, the same procedure generates the same identifier and therefore maps to the same position in the fingerprint. Different substructures can give rise to the same bits, so while a fingerprint can easily confirm the absence of a substructure, it does not indicate any certainty that the substructure is indeed present. These **hash collisions** are what you see in spurious substructures in CAS SciFinder.

> **Morgan fingerprints describe which local molecular environments occur within a molecular graph.** Unlike predefined structural keys, those environments do not need to be chosen before the molecules are analysed, making circular fingerprints useful across a wide range of chemical datasets.

A binary fingerprint records only whether a feature occurs:

$$x_i \in 0, 1. \tag{1.6}$$

An alternative is a **count fingerprint**, in which each position records how many times a feature occurs within the molecule:

$$x_i \in 0, 1, 2, \ldots. \tag{1.7}$$

Count fingerprints therefore retain information that is lost when a bit is simply

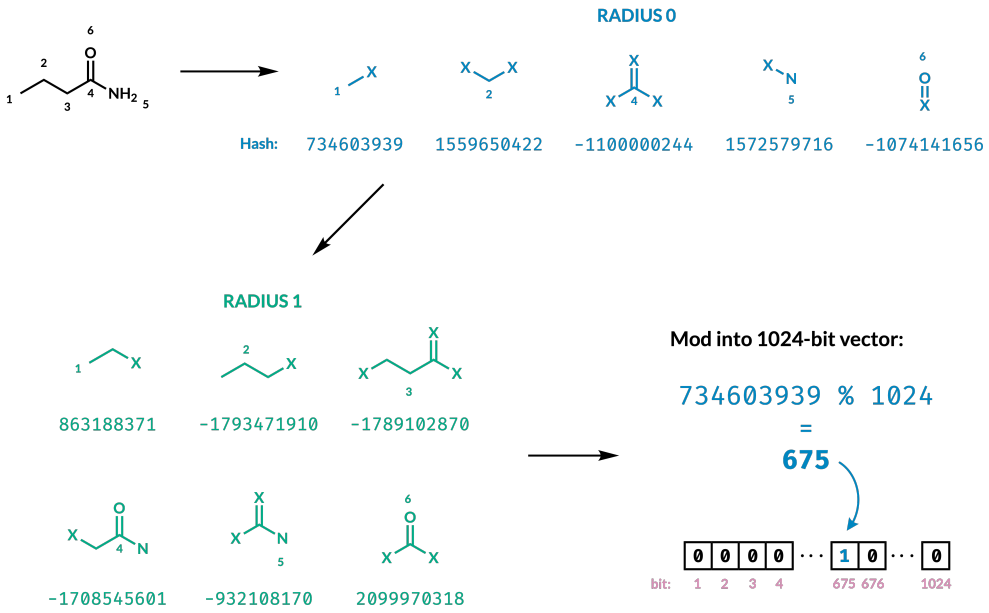



**Figure 1.9.** Generation of an ECFP fingerprint with contributions from environments at different radii. Bits activated by each environment are based on the hashing function.

switched on. Extended-connectivity fingerprints normally distinguish atomic environments using atom-specific information. A related representation, the **Functional-Class Fingerprint (FCFP)**, instead groups atoms according to chemically relevant roles, such as hydrogen-bond donor or acceptor, aromatic character, or membership of broader functional classes.

## 1.7. Summary

In this chapter, we have seen that a molecule can be represented digitally in many different ways, and that **there is no universally 'best' molecular representation**. Instead, the appropriate representation depends on the chemical information that must be retained and the task we want to perform. Cartesian coordinates provide detailed information about **molecular geometry**, while molecular graphs abstract a structure into its **atoms and connectivity**. Line representations such as SMILES provide a compact way of encoding molecular graphs as text, while InChI and InChIKey provide standardised approaches to **chemical identification and data interoperability**. Finally, molecular fingerprints deliberately discard some structural information to produce compact numerical representations that are efficient to store and compare. These representations therefore illustrate a recurring theme in digital chemistry: **the choice of representation determines which chemical information is made explicit, which information is hidden, and which information is lost**. Having learnt how to represent individual molecules, we can now ask a different question: if molecules are represented computationally, **how can we decide how similar or different they are?** Answering this question takes us from individual molecular representations to the organisation and exploration of the **chemical space**.